

Question 2: Transition-Based Dependency Parser

Design Choices and Results

Group 21: Krishna Jhanwar (12341250), Siddharth Rai (12342050), Slok Tulsyan (12342080),
Vasu Garg (12342330), Gaurav Kumar (12340790)

September 8, 2026

1 Overview

An arc-standard transition-based dependency parser was implemented from scratch and trained on the UD_English-EWT treebank (`en_ewt-ud-train.conllu` for training, `en_ewt-ud-dev.conllu` for evaluation). The parser uses a stack, a buffer, and a set of built arcs, and applies one of three moves at each step: **SHIFT**, **LEFT-ARC(label)**, or **RIGHT-ARC(label)**. A scikit-learn classifier trained on POS-tag features decides which move to apply.

2 Data Processing

A CoNLL-U reader parses each sentence into a list of tokens (id, form, UPOS tag, head, dependency label). Lines describing multiword token ranges (e.g. 3-4) and empty nodes (e.g. 3.1) are skipped, since they fall outside the basic dependency tree structure. Dependency labels are simplified by dropping any subtype after a colon (e.g. `obl:tmod` $\rightarrow$ `obl`), which keeps the number of distinct classes the classifier has to learn at a manageable size while preserving the core grammatical relation.

3 Oracle Simulator

The oracle replays the arc-standard parsing process on a sentence whose gold tree is already known, and at each configuration works out the single correct transition. With `s1` as the top of the stack and `s2` the second item:

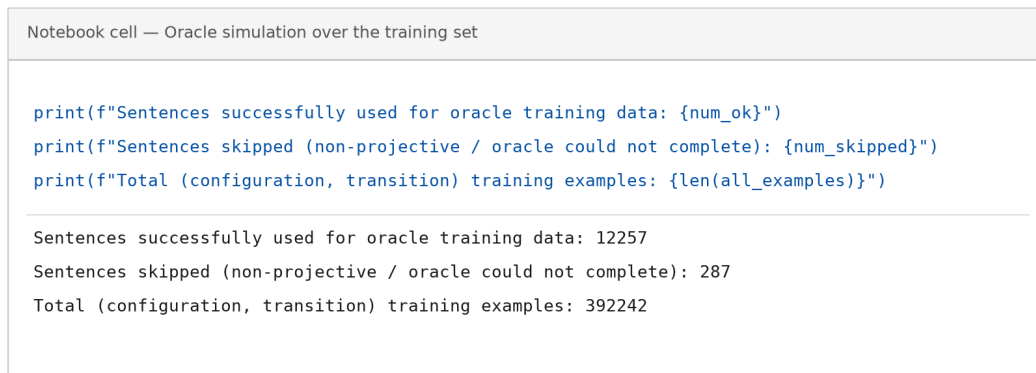
- If `s1` is the gold head of `s2`, and `s2` has no children still waiting in the buffer, apply **LEFT-ARC**.
- Else if `s2` is the gold head of `s1`, and `s1` has no children still waiting in the buffer, apply **RIGHT-ARC**.
- Otherwise apply **SHIFT**.

The “no children waiting” condition prevents a word from being popped off the stack before all of its dependents have been attached — otherwise those dependents would never be reachable again.

The arc-standard system can only construct *projective* trees (no crossing arcs). A small fraction of real sentences are non-projective, so the oracle gets stuck on them; those sentences are simply

skipped when building training data. Out of 12,544 training sentences, 12,257 were used and 287 were skipped for this reason.

Running the oracle over the training set produced 392,242 (configuration, transition) training examples.

The image shows a screenshot of a Jupyter Notebook cell. The title bar of the cell reads "Notebook cell — Oracle simulation over the training set". The cell contains three lines of Python code:

```
print(f"Sentences successfully used for oracle training data: {num_ok}")  
print(f"Sentences skipped (non-projective / oracle could not complete): {num_skipped}")  
print(f"Total (configuration, transition) training examples: {len(all_examples)}")
```

 Below the code, the output of the cell is displayed:

```
Sentences successfully used for oracle training data: 12257  
Sentences skipped (non-projective / oracle could not complete): 287  
Total (configuration, transition) training examples: 392242
```

Figure 1: Notebook output after running the oracle simulator over all training sentences.

4 Feature Extraction

Features are kept deliberately minimal — just the POS tag of:

- the word on top of the stack,
- the second word on the stack (if it exists),
- the first word in the buffer (if it exists),
- the second word in the buffer (if it exists).

A placeholder value "NONE" is used whenever a position does not exist. No word-form or lexical features are used.

5 Model Training

Each transition is encoded as a single class label that combines the move and the dependency label, e.g. SHIFT, LEFT-ARC-det, RIGHT-ARC-obj. A single classifier therefore predicts both the structural move and the grammatical relation together. The POS-tag feature dictionaries are one-hot encoded with `DictVectorizer`, and a `LogisticRegression` classifier is trained on the resulting sparse matrix (392,242 examples $\times$ 73 features, 61 distinct classes). `LogisticRegression` was chosen for being fast, simple, and a standard baseline for this kind of multi-class classification — training completes in about a minute on a normal laptop CPU, with no GPU required.

```

Notebook cell — Feature vectorization and classifier training

vec = DictVectorizer(sparse=True)
X = vec.fit_transform(X_dicts)
clf = LogisticRegression(max_iter=200)
clf.fit(X, y)

Feature matrix shape: (392242, 73)
Training time: 60.8 seconds
Training accuracy (per-transition, not the same as parser accuracy): 0.8130

```

Figure 2: Notebook output for feature vectorization and classifier training.

6 Parser and Legal-Move Filtering

At inference time there is no gold tree to check against, so the parser must ensure every applied move is legal in the current configuration:

- **SHIFT** is legal only if the buffer is non-empty.
- **LEFT-ARC** / **RIGHT-ARC** are legal only if the stack has at least 2 words.
- **LEFT-ARC** is additionally illegal if the second stack item is **ROOT** (**ROOT** can never be a dependent).

At each step, all transition classes are ranked by the classifier’s confidence score, and the highest-scoring class that is legal is applied — rather than blindly applying the top prediction, which could be illegal in the current configuration.

7 Evaluation

The trained parser was run on the full dev set (2,001 sentences, 25,148 words), and the predicted head/label for every word was compared against the gold annotation:

Metric	Score
UAS (Unlabeled Attachment Score)	0.6714
LAS (Labeled Attachment Score)	0.5815

UAS measures how often the predicted head is correct; LAS additionally requires the dependency label to match, so it is stricter and is always lower than UAS.

```
Notebook cell — LAS / UAS evaluation on the dev set

for sent in dev_sents:
    pred_arcs = parse_sentence(sent, clf, vec)
    ...

Evaluated on 2001 dev sentences (25148 words) in 9.8 seconds
UAS (Unlabeled Attachment Score): 0.6714
LAS (Labeled Attachment Score): 0.5815
```

Figure 3: Notebook output for the LAS/UAS evaluation on the dev set.

8 Example Sentences

The parser was also tested on a few hand-picked sentences (manually POS-tagged with the Universal POS tagset), producing sensible dependency trees, e.g. for “*The cat sat on the mat.*” the verb **sat** is correctly identified as the root, with **cat** attached as its subject and **mat** attached as an oblique argument.

```
Notebook cell — Parsing the example sentences

for text, pairs in example_sentences.items():
    arcs = parse_sentence(make_sentence(pairs), clf, vec)
    print_parse(sent_tokens, arcs)

Sentence: The cat sat on the mat.
The      <- cat      (det)
cat       <- sat      (nsubj)
sat       <- ROOT      (root)
on        <- mat      (case)
the       <- mat      (det)
mat       <- sat      (obl)
.         <- sat      (punct)

Sentence: She eats a green salad.
She       <- eats      (nsubj)
eats      <- ROOT      (root)
a         <- salad      (det)
green     <- salad      (amod)
salad     <- eats      (obl)
.         <- eats      (punct)

Sentence: I saw the man with a telescope.
I         <- saw        (nsubj)
saw       <- ROOT      (root)
the       <- man        (det)
man       <- saw        (obl)
with      <- telescope   (case)
a         <- telescope   (det)
telescope <- man        (nmod)
.         <- saw        (punct)
```

Figure 4: Notebook output showing the predicted dependency arcs for the three example sentences.

9 Discussion

- Because the feature set is intentionally small (4 POS tags only, no word forms, no lookahead beyond 2 buffer positions, no stack history), the parser cannot see lexical cues such as a specific verb’s typical argument structure. This limits how high UAS/LAS can go relative to parsers using richer features (e.g. word embeddings), but reflects the ceiling achievable with a purely POS-tag-based arc-standard parser.
- LAS being noticeably lower than UAS is expected: predicting the correct head is an easier decision than additionally predicting the exact dependency label out of 61 possible transition

classes, especially once label subtypes are collapsed and the classifier only sees POS tags rather than lexical context.

- Errors compound over the course of parsing a sentence: an early wrong SHIFT/ARC decision changes the entire remaining stack/buffer configuration, so mistakes tend to cascade — a general property of greedy transition-based parsing, as opposed to globally optimal parsing algorithms.

10 Conclusion

A working arc-standard transition-based dependency parser was built end to end: CoNLL-U reading, oracle-based training data generation, POS-tag feature extraction, a scikit-learn classifier, a legality-aware parsing loop, and LAS/UAS evaluation on the UD_English-EWT dev set, achieving a final LAS of **0.5815** and UAS of **0.6714**.